Univalence Walkthrough

Detailed Technical Design for `cubical-viz`

Oguri Cap

March 2026

Contents

1 Goal and Scope

This document is the implementation specification for the **Univalence Walkthrough** feature of `cubical-viz`. It is a self-contained multi-stage guided proof that shows how the Univalence principle is *proved* (not assumed) in cubical type theory using Glue types.

Target audience: Students who have already explored the Interval, Path Type, Transport, Kan Filling, and Glue Types pages.

Deliverable: A new sidebar entry "Univalence" under the THEORY category, routing to a multi-stage interactive page. Each of the four stages is a full-height panel with a Three.js scene, a KaTeX proof fragment, and a step panel.

2 Mathematical Background

2.1 The Univalence Principle

In homotopy type theory, the *univalence axiom* asserts:

$$\mathrm{ua} : (A \simeq B) \to (A =_{\mathcal{U}} B)$$

that equivalent types are equal. In cubical type theory this is not an axiom but a *theorem*, proved by constructing a path in the universe $\mathcal{U}$ using **Glue types**.

2.2 The Proof in One Paragraph

Given an equivalence $e : A \simeq B$, we construct a line $\mathrm{ua}(e) : \mathcal{U}$ parameterized by $i : \mathbb{I}$ as follows:

$$\mathrm{ua}(e)(i) \;=\; \mathrm{Glue}\big[\, i{=}0 \mapsto (A,\, e),\;\; i{=}1 \mapsto (B,\, \mathrm{id}_B) \,\big]\; B$$

At $i = 0$ this Glue type reduces definitionally to A; at $i = 1$ it reduces to B. Hence $\mathrm{ua}(e)$ is a path $A =_{\mathcal{U}} B$. Furthermore, transport along this path computes as application of e: $\mathrm{transp}^i\,(\mathrm{ua}(e)(i))\, a = e(a)$.

2.3 Key Judgments Per Stage

Stage	Name	Key judgment
1	Setup	$\Gamma \vdash e : A \simeq B$
2	Glue line	$\Gamma,\, i{:}\mathbb{I} \vdash \mathrm{Glue}[\varphi \mapsto (T, f)]\, B : \mathcal{U}$
3	Path in $\mathcal{U}$	$\Gamma \vdash \mathrm{ua}(e) : A =_{\mathcal{U}} B$
4	Transport	$\Gamma \vdash \mathrm{transp}^i(\mathrm{ua}(e)(i))\, a \equiv e(a) : B$

3 Page Architecture

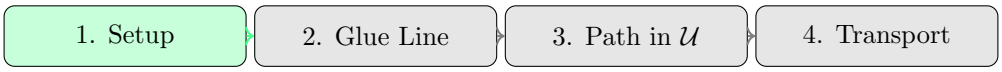
3.1 Route and Component Tree

```
App.svelte
  {:else if current === 'univalence'}
    <Univalence />          // top-level orchestrator

Univalence.svelte
  <StageNav />              // progress bar / stage tabs
  {#if stage === 0} <UniSetup />
  {#if stage === 1} <UniGlue />
  {#if stage === 2} <UniPath />
```

```
  {#if stage === 3} <UniTransport />

Each stage component (e.g. UniSetup.svelte):
  <Rule rule={stageRule} />     // reuses Rule.svelte wrapper
```

3.2 StageNav Component

A horizontal progress bar above the content area:



The active stage is highlighted in green. Completed stages are cyan. Future stages are gray. Clicking a completed stage navigates back.

A "Continue →" button at the bottom-right of the step panel advances to the next stage (enabled only when the user reaches the last step of the current stage).

3.3 State Management

```
// in Univalence.svelte
let stage = $state(0);            // 0-3
let stageCompleted = $state([false, false, false, false]);

function advanceStage() {
  stageCompleted[stage] = true;
  stage = Math.min(stage + 1, 3);
}
```

4 Stage 1: Setup

4.1 Concept

Introduce the two types A and B and the equivalence $e : A \simeq B$. Recall that an equivalence is a function $f : A \to B$ with a contractible fiber — equivalently, a homotopy equivalence.

4.2 KaTeX Judgment

$$\frac{\Gamma \vdash A : \mathcal{U} \qquad \Gamma \vdash B : \mathcal{U} \qquad \Gamma \vdash e : A \simeq B}{\Gamma \vdash \mathrm{ua}(e) : A =_{\mathcal{U}} B} \ (\mathrm{ua})$$

4.3 3D Scene

- Two large spheres (radius 0.9):
  - A at $(-3, 0, 0)$, color `INTERVAL` (0x44DDFF).
  - B at $(+3, 0, 0)$, color `RESULT` (0xFF8844).
- A **curved arrow** from A to B using a `QuadraticBezierCurve3` with control point $(0, 2, 1)$: gradient cyan → orange, with an animated arrowhead traveling along the curve.
- Label sprites: "A" above the left sphere, "B" above the right sphere, "e" midway on the arc.
- A faint dotted horizontal line segment between A and B (dim white, opacity 0.2), labeled "$i \in [0, 1]$" — hinting at the path to be constructed.
- Camera: $(0, 1, 9)$ looking at $(0, 0, 0)$.

4.4 Steps

1. **Two types** — "A and B are types in the universe $\mathcal{U}$. In the univalence proof, we start with an equivalence $e : A \simeq B$." *(timeRange: [0, 0.4])*

2. **Equivalence** — "An equivalence $e : A \simeq B$ is a function $f : A \to B$ with homotopy inverse. It witnesses that A and B have the same structure." *(timeRange: [0.4, 0.7])*

3. **Goal** — "We want to produce a *path* $\mathrm{ua}(e) : A =_{\mathcal{U}} B$ — an element of the identity type in the universe. This path does not exist in MLTT without an axiom; in cubical TT we construct it." *(timeRange: [0.7, 1.0])*

4.5 Term Mappings

termKey	3D object
`A`	Left sphere
`B`	Right sphere
`e`	Arc line

5 Stage 2: Glue Line

5.1 Concept

Construct the Glue type indexed by $i : \mathbb{I}$. At each i, the type is $\mathrm{Glue}[\, i{=}0 \mapsto (A, e),\ \ i{=}1 \mapsto (B, \mathrm{id})\,]\, B$. This is a *line of types* in $\mathcal{U}$.

5.2 KaTeX Judgment

$$\Gamma,\, i{:}\mathbb{I} \ \vdash\ \underbrace{\mathrm{Glue}\big[\, i{=}0 \mapsto (A, e),\ \ i{=}1 \mapsto (B, \mathrm{id})\,\big]\, B}_{\mathrm{ua}(e)(i)} : \mathcal{U}$$

5.3 3D Scene

The scene transitions from Stage 1's two-sphere layout into a **cylinder** connecting A to B.

- The curved arc from Stage 1 thickens into a `CylinderGeometry` (radiusTop 0.25, radiusBottom 0.25, height 6) between $(-3, 0, 0)$ and $(3, 0, 0)$, oriented along the X-axis. Color: gradient texture from cyan (left) to orange (right). In Three.js, simulate the gradient by two `MeshBasicMaterial` half-cylinders, or use vertex colors on a `TubeGeometry` following the path.

- A thin **slider sphere** (radius 0.18, color white) moves along the cylinder at $x = \mathrm{lerp}(-3, 3, t)$, representing the current value of i.

- **End cap labels**: "Glue(0) $\equiv A$" at the left end; "Glue(1) $\equiv B$" at the right end.

- **Cross-section indicator**: At the slider position, draw a thin circle (torus, radius 0.28, tube 0.02) perpendicular to the cylinder axis, color `FILL_RESULT` (0xFF44AA). A text sprite above it reads "Glue(i)".

- The two original spheres (A and B) remain visible at the ends, slightly smaller (radius 0.7).

- Camera: $(0, 2, 9)$ looking at $(0, 0, 0)$.

5.4 Steps

1. **Glue type** — "A Glue type bundles a base type B with partial data specifying how certain faces map to other types via equivalences. Here, the face i=0 is glued to A via e." *(timeRange: [0, 0.35])*

2. **Line of types** — "For each $i : \mathbb{I}$, the Glue type gives a type $\mathrm{ua}(e)(i)$. The slider shows i varying: at the left end the type is definitionally A, at the right it is B." *(timeRange: [0.35, 0.7])*

3. **Boundary conditions** — "The definitional equalities $\mathrm{Glue}(0) \equiv A$ and $\mathrm{Glue}(1) \equiv B$ are enforced by the Glue type former's reduction rules — no propositional proof required." *(timeRange: [0.7, 1.0])*

5.5 Term Mappings

termKey	3D object
`glue_line`	Cylinder/tube mesh
`A_end`	Left sphere
`B_end`	Right sphere
`glue_i`	Slider sphere + torus cross-section

6 Stage 3: Path in the Universe

6.1 Concept

The line of Glue types *is* a path in $\mathcal{U}$. Show the universe as an ambient space and display the path $\mathrm{ua}(e)$ living inside it.

6.2 KaTeX Judgment

$$\Gamma \vdash \lambda\, i.\ \mathrm{Glue}[\cdots]\, B \ :\ A =_{\mathcal{U}} B$$

6.3 3D Scene

- **Universe plane**: a large `GridHelper` at $y = -2.5$, size 20, divisions 20, colors `0x222244`/`0x333366`. This represents $\mathcal{U}$ as an ambient "ground plane".

- **Path curve**: the cylinder from Stage 2 is re-rendered as a thinner curved line (the path $\mathrm{ua}(e)$), floating above the universe plane. Use a `QuadraticBezierCurve3` from $(-3, 0, 0)$ to $(3, 0, 0)$ with control point $(0, 1.5, 1.5)$, gradient cyan→orange.

- **Type labels**: Small floating text boxes labeled "A" and "B" at the two ends, connected to the path endpoints by thin vertical lines to the grid below.

- **Path label**: "$\mathrm{ua}(e)$" as a text sprite above the midpoint of the arc.

- **Camera**: zoomed out to $(0, 4, 14)$ looking at $(0, 0, 0)$, giving a panoramic view of the "universe".

6.4 Camera Transition

When Stage 3 mounts, animate the camera from its Stage 2 position $(0, 2, 9)$ to $(0, 4, 14)$ over 1.5 seconds using a `TWEEN`-style lerp:

```
// In onMount / setup:
const startPos = new THREE.Vector3(0, 2, 9);
const endPos   = new THREE.Vector3(0, 4, 14);
// lerp over elapsed 0..1.5s during first 1.5s of animation
```

6.5 Steps

1. **The universe $\mathcal{U}$** — "The universe $\mathcal{U}$ contains all types as points. A path in $\mathcal{U}$ between A and B is exactly an element of $A =_{\mathcal{U}} B$." *(timeRange: [0, 0.4])*

2. **ua(e) as a path** — "The line of Glue types $i \mapsto \mathrm{Glue}[\cdots]B$ is a *term* of type $A =_{\mathcal{U}} B$. It is a literal path in the universe from A to B." *(timeRange: [0.4, 0.75])*

3. **Univalence achieved** — "This is the univalence map: $\mathrm{ua} : (A \simeq B) \to (A =_{\mathcal{U}} B)$. No axiom was needed — the path was constructed from the Glue type former." *(timeRange: [0.75, 1.0])*

7 Stage 4: Transport

7.1 Concept

Verify that transporting along $\mathrm{ua}(e)$ computes as application of e. This is the *computation rule* for univalence: $\mathrm{transp}^i\,(\mathrm{ua}(e)(i))\,a \equiv e(a)$.

7.2 KaTeX Judgment

$$\Gamma \;\vdash\; \mathrm{transp}^i\big(\mathrm{ua}(e)(i)\big)\,a \;\equiv\; e(a) \;:\; B$$

7.3 3D Scene

A split composition: the left half shows the path $\mathrm{ua}(e)$ (from Stage 3, simplified); the right half shows the transport computation.

- **Left panel** (world $x < 0$, i.e. the path in $\mathcal{U}$):
  - Small versions of A (cyan sphere, radius 0.5, at $(-3, 0, 0)$) and B (orange sphere, radius 0.5, at $(-0.5, 0, 0)$).
  - The path arc between them (gradient, thin).
  - A moving point on the arc, colored white.
- **Right panel** (world $x > 0$, i.e. the fiber):
  - A small cyan sphere labeled "$a : A$" at $(1, 0, 0)$.
  - An animated **morphing dot**: starts at $(1, 0, 0)$ (position in A, cyan) and travels along a Bézier arc to $(3.5, 0, 0)$ (position in B, orange), driven by the same t as the path traversal.
  - Label: "$e(a)$" appears at the orange end as the dot arrives.
  - A thin vertical **separator line** at $x = 0$ from $y = -2$ to $y = 2$, color dim white.
- Camera: $(0, 2, 10)$ looking at $(0, 0, 0)$.

7.4 Steps

1. **Recall transport** — "Transport along a path $p : A =_{\mathcal{U}} B$ sends elements of A to elements of B: $\mathrm{transp}^i\, p(i)\, a : B$." *(timeRange: [0, 0.35])*

2. **Transport along ua(e)** — "Transporting $a : A$ along $\mathrm{ua}(e)$ moves a from the fiber at $i = 0$ (type A) to the fiber at $i = 1$ (type B)." *(timeRange: [0.35, 0.7])*

3. **Computation rule** — "By the definitional reduction of Glue types, this transport *computes* to $e(a)$: the underlying function of the equivalence. This is the *transport computation rule* for univalence." *(timeRange: [0.7, 1.0])*

7.5 Completion

After reaching the final step of Stage 4, a **completion overlay** appears:

- Semi-transparent dark backdrop over the 3D scene.
- Centered panel:
  - Header: "Univalence Proved! "
  - Summary: the full chain $e \mapsto \mathrm{ua}(e) \mapsto \mathrm{transp}^i(\mathrm{ua}(e)(i)) \equiv e$ displayed in KaTeX.
  - Two buttons: "Review from Stage 1" (resets stage to 0) and "Explore related rules" (links to Glue Types and Transport pages).

8 Component Specifications

8.1 Univalence.svelte

```
<script lang="ts">
  import StageNav from ./univalence/StageNav.svelte';
  import UniSetup from ./univalence/UniSetup.svelte';
  import UniGlue from ./univalence/UniGlue.svelte';
  import UniPath from ./univalence/UniPath.svelte';
  import UniTransport from ./univalence/UniTransport.svelte';

  let stage = $state(0);
  let completed = $state([false,false,false,false]);

  function advance() {
    completed[stage] = true;
    if (stage < 3) stage++;
  }
</script>

<div class="univalence-page">
  <StageNav {stage} {completed} onSelect={(s) => stage = s} />
  <div class="stage-content">
    {#if stage === 0}<UniSetup onComplete={advance} />{/if}
    {#if stage === 1}<UniGlue onComplete={advance} />{/if}
    {#if stage === 2}<UniPath onComplete={advance} />{/if}
    {#if stage === 3}<UniTransport onComplete={advance} />{/if}
  </div>
</div>
```

8.2 StageNav.svelte

Props: `stage: number`, `completed: boolean[]`, `onSelect: (n: number) => void`.

Renders four buttons. A button for stage k is:

- *Active* (`stage === k`): green background.
- *Completed* (`completed[k]`): cyan background, clickable.
- *Future*: gray, disabled.

8.3 UniSetup / UniGlue / UniPath / UniTransport

Each wraps a `RuleDefinition` (same interface as all other rules) and passes it to `Rule.svelte`. Additionally each accepts an `onComplete: () => void` callback, which it forwards to the "Continue →" button that replaces the usual "Next →" at the final step.

The `RuleDefinition` for each stage:

```
const stageRule: RuleDefinition = {
  id:          'univalence-setup',        // unique id
  name:        'Setup',
  katex:       '...',                     // judgment string
  description: '...',
  setup:       (scene, camera) => { ... },
  update:      (t, elapsed) => { ... },
  cleanup:     (scene) => { ... },
  steps:       [ ... ],                   // 3 steps per stage
  termMappings: { ... },
};
```

8.4 Completion Overlay

Implemented inside `UniTransport.svelte`. Triggered when the user reaches step 3 and the animation completes one cycle:

```
{#if showCompletion}
  <div class="completion-overlay">
    <div class="completion-panel">
      <h2>Univalence Proved!     </h2>
      <!-- KaTeX summary -->
      <div class="summary">{@html katexSummary}</div>
      <button onclick={() => stage = 0}>Review from Stage 1</button>
      <button onclick={() => navigate('glue-type-rule')}>
        Explore: Glue Types
      </button>
    </div>
  </div>
{/if}
```

9 File Structure

```
src/lib/
  univalence/
    Univalence.svelte        # orchestrator
    StageNav.svelte          # progress bar
    UniSetup.svelte          # stage 1
```

```
    UniGlue.svelte            # stage 2
    UniPath.svelte            # stage 3
    UniTransport.svelte       # stage 4

src/lib/types.ts              # add 'univalence' to VisualizationType,
                              # add to THEORY category
src/App.svelte                # add render branch (isStandalone)
```

10 Implementation Order for Copilot

This is a large task that should be split into two Copilot sessions to stay within context limits.

Session A — Scaffolding + Stages 1 & 2.

1. Create `src/lib/univalence/` directory.
2. Implement `StageNav.svelte`.
3. Implement `Univalence.svelte` orchestrator with empty stage stubs.
4. Register `'univalence'` in `types.ts` and `App.svelte`.
5. Implement `UniSetup.svelte` (Stage 1: two spheres, arc, 3 steps).
6. Implement `UniGlue.svelte` (Stage 2: cylinder, slider, 3 steps).
7. Build passes. Commit: "Univalence: scaffold + stages 1-2".

Session B — Stages 3, 4 & Completion.

1. Implement `UniPath.svelte` (Stage 3: universe grid, zoomed-out path).
2. Implement `UniTransport.svelte` (Stage 4: split scene, morphing dot).
3. Add completion overlay to `UniTransport.svelte`.
4. Final build pass and commit: "Univalence: stages 3-4 + completion overlay".

11 Color Summary

Concept	Constant	Hex
Type A	`INTERVAL`	`0x44DDFF`
Type B	`RESULT`	`0xFF8844`
Equivalence e	gradient INTERVAL→RESULT	—
Glue line / path ua(e)	gradient INTERVAL→RESULT	—
Cross-section Glue(i)	`FILL_RESULT`	`0xFF44AA`
Universe grid	`0x222244/0x333366`	—
Transport dot start	`INTERVAL`	`0x44DDFF`
Transport dot end	`RESULT`	`0xFF8844`
Stage: active	`PARTIAL_DATA`	`0x44FF88`
Stage: completed	`INTERVAL`	`0x44DDFF`
Stage: future	gray	`0x555566`

12 Risks and Mitigations

Risk	Likelihood	Mitigation
Copilot generates incorrect Three.js for cylinder gradient	Medium	Use two half-cylinders with solid colors (INTERVAL / RESULT) instead of a gradient material
Camera transition causes jank when switching stages	Low	Skip camera animation in first version; set camera directly in `setup()`
Completion overlay KaTeX string too long	Low	Use `displayMode: true` and wrap in a scrollable div
`Rule.svelte` doesn't pass `onComplete` through	Medium	Pass it as a prop to the `StepPanel`; or handle in each stage component by watching `activeStep === rule.steps.length - 1`